

1. Write a Pandas program to create a) Date time object for Jan 15 2012. b) Specific date and time of 9:20 pm. c) Local date and time. d) A date without time. e) Current date. f) Time from a date time. g) Current local time.

```
import pandas as pd
from datetime import datetime

# a) Date time object for Jan 15 2012.
a = pd.to_datetime("2012-01-15")

# b) Specific date and time of 9:20 pm.
b = pd.Timestamp("2023-01-01 21:20:00")

# c) Local date and time.
c = pd.Timestamp.now()

# d) A date without time.
d = pd.to_datetime("2023-01-01").date()

# e) Current date.
e = pd.to_datetime("today").date()

# f) Time from a date time.
f = pd.Timestamp("2023-01-01 21:20:00").time()

# g) Current local time.
g = c.time()

print(a)
print(b)
print(c)
print(d)
print(e)
print(f)
print(g)
```

2. Write a Pandas program to convert all the string values to upper, lower cases in a given pandas series. Also find the length of the string values. s = pd.Series(['X', 'Y', 'T', 'Aaba', 'Baca', 'CABA', None, 'bird', 'horse', 'dog'])

```
import pandas as pd

s = pd.Series(['X', 'Y', 'T', 'Aaba', 'Baca', 'CABA', None, 'bird',
               'horse', 'dog'])

# Convert to upper case
upper_case = s.str.upper()

# Convert to lower case
lower_case = s.str.lower()

# Length of each string
lengths = s.str.len()

print(upper_case)
print(lower_case)
print(lengths)
```

3. After accidentally leaving an ice chest of fish and shrimp in your car for a week while you were on vacation, you're now in the market for a new vehicle. Your insurance didn't cover the loss, so you want to make sure you get a good deal on your new car. Given a Series of car asking_prices and another Series of car fair_prices, determine which cars for sale are a good deal. In other words, identify cars whose asking price is less than their fair price. The result should be a list of integer indices corresponding to the good deals in asking_prices.

```
import pandas as pd

asking_prices = pd.Series([5000, 7000, 12000, 15000])
fair_prices = pd.Series([5500, 7200, 11000, 15000])

# Find good deals
good_deals = asking_prices[asking_prices < fair_prices].index.tolist()
```

```
for x in good_deals:

    print(f"index: {x} Asking price: {asking_prices[x]}")
```

4. Whenever your friends John and Judy visit you together, y'all have a party. Given a DataFrame with 10 rows representing the next 10 days of your schedule and whether John and Judy are scheduled to make an appearance, insert a new column called `days_til_party` that indicates how many days until the next party. `days_til_party` should be 0 on days when a party occurs, 1 on days when a party doesn't occur but will occur the next day, etc.

```
import pandas as pd

schedule = pd.DataFrame({
    'John': [False, False, True, False, True, False, False, True, False,
False],
    'Judy': [False, True, True, False, True, False, False, True, False,
True]
})

# Party occurs when both John and Judy are present
party_days = (schedule['John'] & schedule['Judy'])

# Calculate days until next party
days_til_party = []
for i in range(len(schedule)):
    future = party_days[i:]
    til = (future.idxmax() - i) if future.any() else -1
    days_til_party.append(til)

schedule['days_til_party'] = days_til_party

print(schedule)
```

5. Given a dataset of concerts, count the number of concerts per (artist, venue), per year month. Make the resulting table be a wide table - one row per year month with a column for each unique (artist, venue) pair. Use the cross product of the artists and venues Series to determine which (artist, venue) pairs to include in the result.

```

import pandas as pd

concerts = pd.DataFrame({
    'artist': ['A', 'B', 'A', 'C', 'B', 'C'],
    'venue': ['X', 'Y', 'X', 'Z', 'Y', 'Z'],
    'date': pd.to_datetime([
        '2023-01-15', '2023-01-15', '2023-01-20',
        '2023-02-15', '2023-02-15', '2023-02-20'
    ])
})

concerts['year_month'] = concerts['date'].dt.to_period('M')

# Group by year_month, artist and venue and count
grouped = concerts.groupby(['year_month', 'artist', 'venue']).size().reset_index(name='count')

# Create artist_venue identifier
grouped['artist_venue'] = grouped['artist'] + "_" + grouped['venue']

# Pivot table to wide format
result = grouped.pivot_table(index='year_month', columns='artist_venue', values='count',
    fill_value=0)

```